

Trabalho Prático de Sistemas Operacionais

O Trabalho Prático é voltado ao desenvolvimento de habilidades e competências em Programação Concorrente e é composto de três projetos de sistemas concorrentes:

- Sistema DynaThreadMaker
- Sistema CentralTalk
- Sistema PeerTalk

Trabalhem e desenvolvam esses projetos com a instalação do sistema operacional Linux Mint ou Ubuntu no VirtualBox, tal como visto em aula. Da mesma forma que no Trabalho Teórico previamente realizado para a disciplina, o Trabalho Prático pode ser desenvolvido em grupos de três a cinco alunos, preferencialmente o mesmo grupo do Trabalho Teórico.

Como resultado do Trabalho Prático devem ser produzidos sistemas e programas em C capazes de atender os requisitos de projeto de cada um dos sistemas concorrentes especificados neste documento.

A entrega dos resultados do Trabalho Prático é formada por um arquivo ZIP contendo todos os sistemas e programas desenvolvidos para o trabalho e também um relatório em formato de arquivo PDF demonstrando como cada projeto foi desenvolvido. Para esse relatório use o formato de artigo da SBC, com mínimo de 10 páginas.

Sistema *DynaThreadMaker*

O sistema ***DynaThreadMaker*** é um sistema para criação de *threads* simuladas pelo usuário. Este sistema é composto de dois tipos de processos distintos: os processos cliente que implementam a interface de usuário e o processo servidor que executa as requisições de criação de *threads*.

O processo servidor de criação de *threads* deve ser posto em execução antes dos processos clientes de interface de usuário, para tanto basta abrir uma janela de terminal e executar o programa do processo servidor, deixando essa janela posteriormente minimizada com o programa/processo servidor rodando.

Para entrar no sistema um usuário deve abrir uma janela de terminal e executar o programa cliente de interface de usuário. A comunicação entre os clientes de interface de usuário é o único processo servidor criador de *threads* é feita com o mecanismo de IPC de filas de mensagens. Uma única fila de mensagens é necessária para implementar essa comunicação.

A comunicação entre os processos cliente e servidor deve ser implementada por uma fila de mensagens. A figura 1 mostra a arquitetura geral do sistema *DynaThreadMaker*.

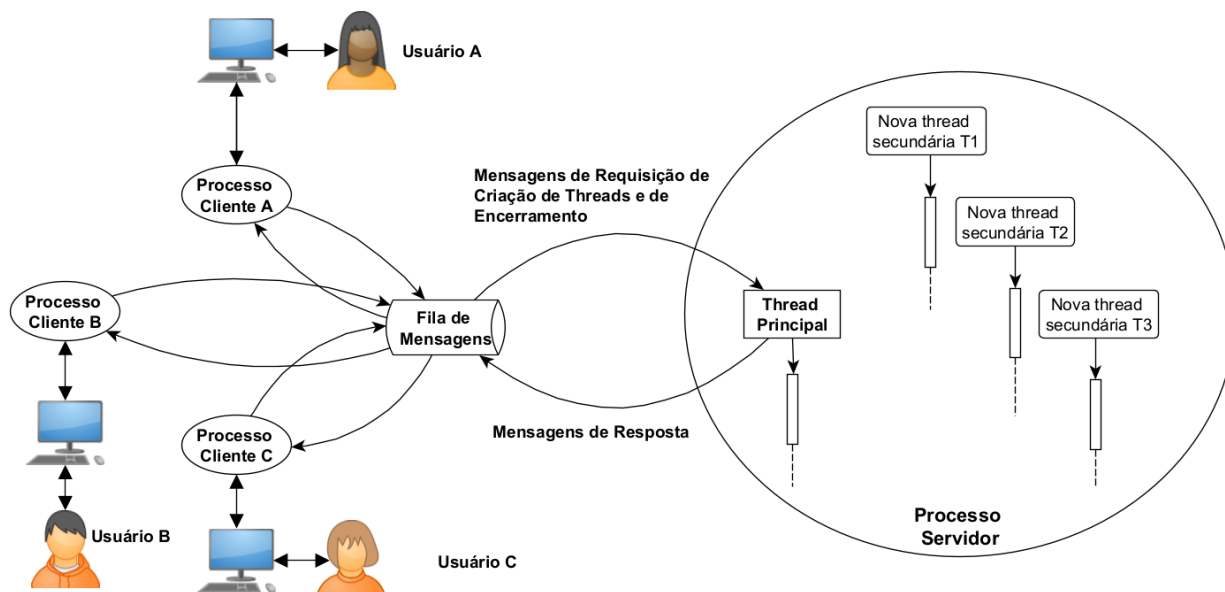


Fig. 1. Diagrama Geral da Arquitetura do Sistema ***DynaThreadMaker***

O campo identificador ou código da mensagem (*msgtyp*) é usado para separar as mensagens destinadas ao servidor das mensagens destinadas a um particular cliente da seguinte forma:

- O código de tipo 1 indica mensagens destinadas ao servidor de criação de *threads*.
- O PID (*Process Identifier*) dos processos cliente indicam mensagens destinadas aos clientes.

Pelo menos dois tipos de mensagens devem ser implementadas no sistema ***DynaThreadMaker***:

- Mensagem de criação de uma nova *thread* que contém como parâmetros, as informações necessárias para a criação da nova *thread*. A especificação de quais são essas informações é definida a seguir.
- Mensagem de encerramento do servidor, enviada por algum processo cliente ao processo servidor indicando que a operação do servidor deve ser finalizada.

O processo servidor cria *threads* de acordo com as seguintes informações transmitidas como campos da mensagem de criação de *threads*:

1. Identificador (PID) do processo cliente: esse campo informa qual processo cliente está fazendo a requisição de criação de *thread*. O valor do PID deste processo deve ser usado nas mensagens de resposta enviadas para esse processo cliente.
2. Nome da nova *thread*: cada *thread* deve ter um nome informado pelo usuário. Quando a *thread* inicia a execução ela mostra uma mensagem com o nome da *thread* e indicando que está iniciando a execução, da mesma forma quando a *thread* estiver finalizando sua execução deve imprimir na tela uma mensagem mostrando seu nome e indicando que está sendo encerrada.
3. Tempo que a nova *thread* vai rodar (quantos segundos): cada *thread* tem um tempo de execução de ***T*** segundos informado pelo usuário. Esse tempo de execução ***T*** é gasto com a primitiva ***sleep(T)*** que simplesmente coloca a *thread* para dormir por ***T*** segundos, liberando a CPU para outras *threads*/processos.
4. Mensagens de texto impressas no início e no fim nova *thread*: o usuário deve informar que mensagem de texto deve ser impressa pela *thread* quando inicia a operação e que mensagem deve ser impressa quando a *thread* finaliza sua execução.

5. Memória dinâmica alocada pela nova *thread*: o usuário também deve informar qual é o tamanho *N* do vetor de inteiros a ser **alocado dinamicamente** e repassado para a *thread*. A *thread* deve inicializar esse vetor com os valores de 1 até *N* e depois imprimir o somatório das posições inicializadas do vetor na tela.

O processo cliente implementa a interface de usuário, onde o usuário diz se quer criar uma nova *thread* ou encerrar a operação do sistema.

Se o usuário selecionar a opção de finalizar a execução do servidor, então o processo cliente envia uma mensagem de encerramento de execução para o processo servidor e depois finaliza a sua própria execução (os demais processos clientes de interface de usuário do sistema *DynaThreadMaker*, se existirem terão que ser encerrados manualmente pelos seus usuários).

Caso o usuário queira criar uma nova *thread*, então são solicitadas ao usuário as seguintes informações, correspondentes aos campos da mensagem de criação de *thread*:

- O nome da nova *thread* a ser criada;
- Quanto tempo a nova *thread* vai rodar (quantos segundos) ;
- Qual mensagem de texto deve ser impressa no início da execução da nova *thread*;
- Qual mensagem de texto deve ser impressa no fim da execução da nova *thread*.
- Quanta memória dinâmica deve ser alocada pela nova *thread*.

Quando o usuário finaliza a entrada dessas informações, o processo cliente monta uma mensagem de criação de nova *thread* com essas informações e envia essa mensagem para o processo servidor de criação de *threads*.

O processo servidor lê continuamente a fila de mensagens esperando requisições. Quando esse processo recebe uma nova mensagem, ele retira as informações da mensagem. Se o processo cliente enviar uma mensagem de encerramento de operação porque o usuário solicitou a finalização do sistema, então o processo servidor simplesmente finaliza sua execução.

Porém, se a mensagem for de criação de uma nova *thread* então o processo servidor extrai os valores dos campos contidos na mensagem para criar uma nova *thread* secundária, passando esses valores como argumentos da nova *thread*.

Entretanto, é importante ter em conta que as variáveis passadas como argumentos para a nova *thread* secundária estão na memória compartilhada por todas as *threads*. Se o processo servidor atender uma nova solicitação, enquanto outra *thread* ainda está executando, essas variáveis poderão ser alteradas causando um comportamento imprevisível e incoerente. Assim a nova *thread* secundária recém criada deve copiar esses argumentos em sua área de variáveis locais antes que outras *threads* possam alterar essas variáveis.

Porém, como não há garantia de sincronismo entre a solicitação de criação de uma *thread* e a execução desta pelo sistema operacional é importante que as variáveis passadas como parâmetros para a *thread* sejam tratadas como uma área crítica que necessita de acesso mutuamente exclusivo. Para tanto, basta que uso dessas variáveis seja controlado por um semáforo binário (um *mutex*) de forma que após serem preenchidas pela *thread* principal e passadas como parâmetro para a *thread* secundária sendo criada, a *thread* principal somente prossegue a execução para tratar novas solicitações quando a *thread* secundária recém criada copiar os dados dos argumentos para suas variáveis locais e sinalizar por *mutex* que isso já foi feito. A figura 2 mostra o diagrama UML de atividades (ou *tasks*) das *threads* do processo servidor do sistema *ThreadMaker*, indicando os eventos de criação de *threads* e de indicação de que a cópia dos argumentos para as variáveis locais foi feito.

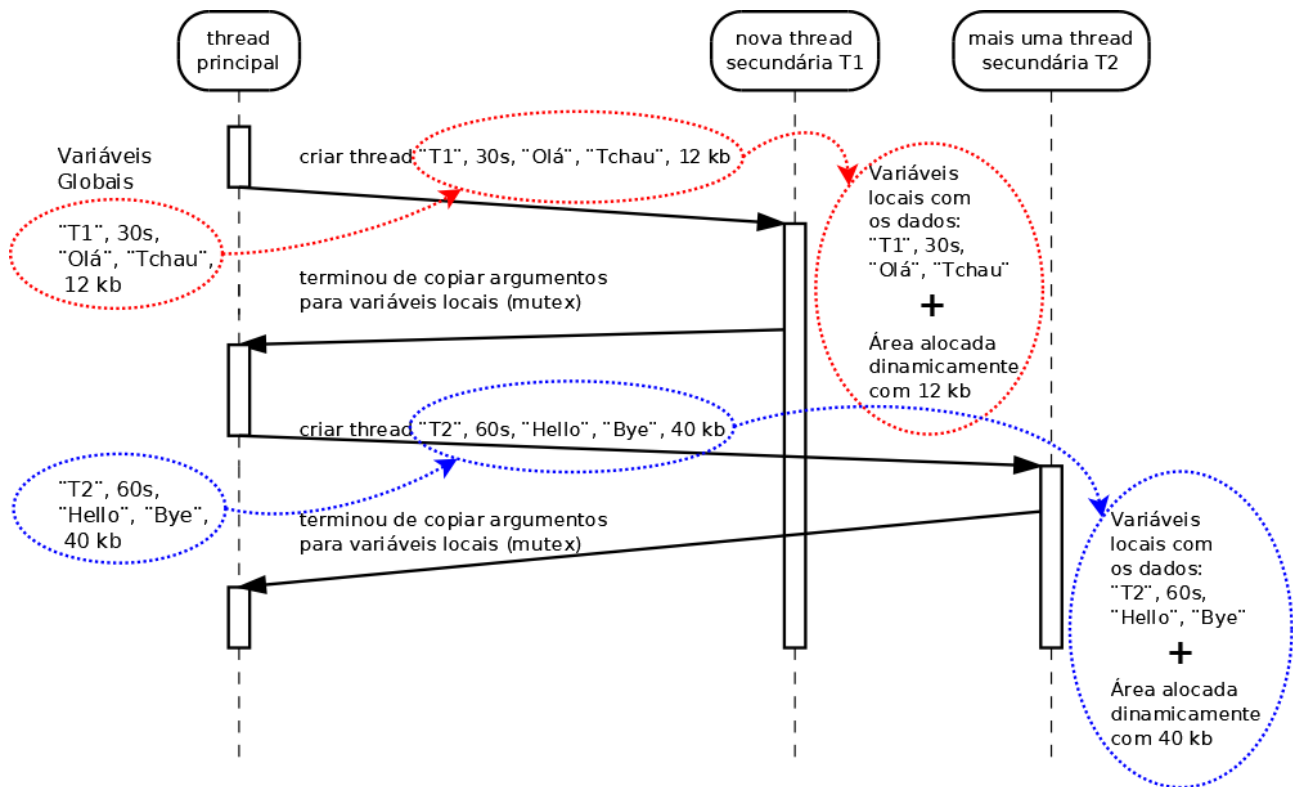


Fig. 2. Diagrama UML de Atividades (ou *Tasks*) das *Threads* do processo servidor

Com os dados de criação da *thread* armazenados nas variáveis locais, a *thread* secundária procede então para atender a solicitação do usuário. Primeiro ela imprime seu nome, a mensagem inicial definida pelo usuário e o número de segundos que irá dormir no terminal onde o executor está rodando. Depois essa *thread* dorme pelo número de segundo definido pelo usuário. Ao final desse tempo a *thread* imprime a mensagem final definida pelo usuário e encerra sua execução.

Logo após criar a *thread* o processo servidor envia uma resposta ao processo cliente pela fila de mensagens avisando que a *thread* foi criada (ou informando que houve algum problema, se isso ocorrer). Depois disso o processo servidor volta a esperar novas solicitações do processo cliente.

O processo cliente após enviar a requisição ao processo servidor, espera pela resposta do processo servidor por um tempo máximo. Se receber a resposta, escreve essa mensagem na tela e depois volta a esperar novas solicitações do usuário.

Se o tempo máximo exceder (houve um *timeout*), o processo cliente avisa o usuário que ocorreu um *timeout* e depois volta a esperar as solicitações do usuário.

Sistema CentralTalk

O sistema **CentralTalk** é um sistema para comunicação e troca de mensagens entre usuários que opera de forma centralizada e é composto de dois tipos de processos:

O sistema **CentralTalk** é um sistema para comunicação e troca de mensagens entre usuários que opera de forma centralizada e é composto de dois tipos de processos:

- Processos clientes, denominados de *speaker*, que implementam a interface de uso do sistema pelos usuários.

- Processo servidor, denominado de *chairman*, que implementa e controla os serviços de comunicação entre os clientes.

Toda a comunicação entre estes processos deve ser implementada pelo mecanismo de IPC de fila de mensagens visto em aula. A figura 3 mostra a arquitetura geral do sistema *CentralTalk*.

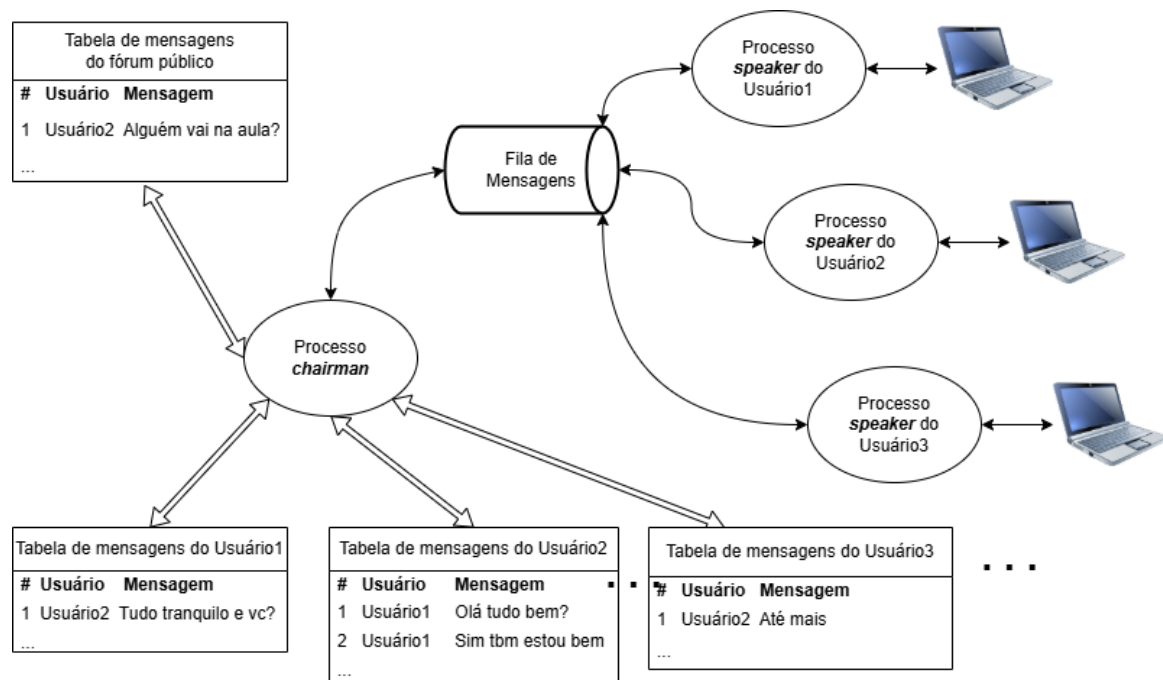


Fig. 3. Diagrama Geral da Arquitetura do Sistema *CentralTalk*

O processo servidor *chairman* deve ser posto em execução antes dos processos clientes, para tanto basta abrir uma janela de terminal e executar o *chairman*, deixando essa janela posteriormente minimizada com o processo *chairman* rodando.

Para entrar no sistema um usuário deve abrir uma janela de terminal e executar o programa cliente *speaker* e se logar no sistema. No login o usuário deve informar seu nome de usuário (não é necessário senha). O nome do usuário é um texto sem espaço em branco de até 20 caracteres. Se não houver nenhum outro usuário já logado com o mesmo nome, o login será aceito.

Para troca de informações entre os diversos clientes *speaker* (um para cada usuário distinto) e o único processo servidor *chairman* utilize o mecanismo de IPC de filas de mensagens. Use o campo identificador ou código da mensagem (*msgtyp*) para separar as mensagens destinadas ao servidor *chairman* das mensagens destinadas a um particular cliente *speaker*. Use o código de tipo 1 para indicar mensagens destinadas ao servidor *chairman* e use o PID (*Process IDentifier*) dos processos cliente para indicar mensagens destinadas aos clientes.

No procedimento de login, o cliente *speaker* solicita que o usuário entre com seu nome e então envia ao servidor *chairman* uma mensagem contendo o nome do usuário que está tentando se logar ao sistema e o PID do cliente *speaker* que está interagindo com esse usuário. Depois o cliente *speaker* espera por uma mensagem de tipo igual ao seu PID com a resposta do servidor *chairman*. Se a resposta for uma confirmação do sucesso do login, então o cliente *speaker* considera que o login teve sucesso e segue com a operação normal do sistema de comunicação. Se a resposta indicar que o processo *chairman* não pode fazer login então o cliente *speaker* repassa essa informação ao usuário, podendo encerrar a operação ou fazer uma nova tentativa de login, se o usuário quiser.

Na operação normal do sistema de comunicação, em todas as interações com o usuário, o cliente *speaker* apenas lê os comandos do usuário e verifica se estão corretamente digitados. Ele não executa estes comandos. Ao invés, ele formata os comandos numa estrutura de dados encapsulada em uma mensagem de

requisição e envia esta mensagem para o processo *chairman* através da fila de mensagens. Note que as mensagens de requisição devem conter o PID do cliente *speaker*. Somente assim o processo *chairman* saberá a qual cliente *speaker* deve enviar a resposta após executar o comando contido na mensagem de requisição.

Os seguintes comandos de usuário são tratados pelo processo servidor *chairman*:

- **send** <nome-usuário> <texto-mensagem> – envia uma mensagem de texto do usuário atual para o usuário destino identificado por <nome-usuário>. A mensagem somente é enviada se o usuário de destino estiver logado no sistema, caso contrário uma mensagem de erro deve ser mostrada na tela. Caso o usuário de destino esteja logado no sistema, a mensagem é na verdade armazenada em uma lista mantida pelo processo *chairman*, que é mostrada ao usuário de destino somente quando ele solicitar ver suas mensagens através do comando **msgs**.
- **msgs** – mostra (lista) na tela todas as mensagens recebidas pelo usuário. Se não houver nenhuma mensagem destinada ao usuário avisa que não há mensagens.
- **post** <texto-mensagem> – publica uma mensagem de texto no fórum público de mensagens.
- **show** – mostra (lista) na tela todas as mensagens que foram publicadas no fórum público de mensagens.
- **del msgs** – remove (deleta) todas as mensagens destinadas ao usuário.
- **del post** <número> – remove (deleta) a mensagem com índice <número> que foi publicada no fórum.
- **del posts** – remove (deleta) todas as mensagens publicadas no fórum.
- **users** – mostra na tela a lista de usuários correntemente logados no sistema.
- **myid** – mostra o nome e identificador (PID) do usuário atual.
- **exit** – sai do sistema **CentralTalk**, mas antes faz o *logout* do usuário.

O servidor *chairman* é o processo do sistema que realmente executa os comandos dos usuários. Assim, conforme pode ser visto na figura 3 o servidor deve manter listas de mensagens recebidas para cada um dos usuários, atualizando as listas de acordo com as requisições dos usuários enviadas pelos processos clientes *speaker*.

Um processo cliente *speaker* apenas lê os comandos do usuário, separa e identifica os campos do comando e envia uma mensagem de requisição ao processo *chairman* para que ele execute o comando. As respostas ocasionadas pela execução do comando são posteriormente repassadas do processo *chairman* para o cliente *speaker* através de uma ou mais mensagens de resposta.

Toda a interação entre os clientes *speaker* e o processo *chairman* ocorre através da fila de mensagens única compartilhada por todos esses processos.

Sistema *PeerTalk*

O sistema **PeerTalk** também é um sistema para comunicação e troca de mensagens entre usuários com uma funcionalidade similar ao sistema **CentralTalk**, porém o sistema **PeerTalk** opera de forma descentralizada não tendo a necessidade de um processo central de controle como o processo servidor *chairman* do sistema **CentralTalk**. Assim o único tipo de componente (processo) do sistema **PeerTalk** é o processo *peertalker*, que interage com outros processos *peertalker*, usando mecanismos de comunicação *peer-to-peer*, para prover os serviços de comunicação entre os usuários. A figura 4 mostra a arquitetura geral do sistema *PeerTalk*.

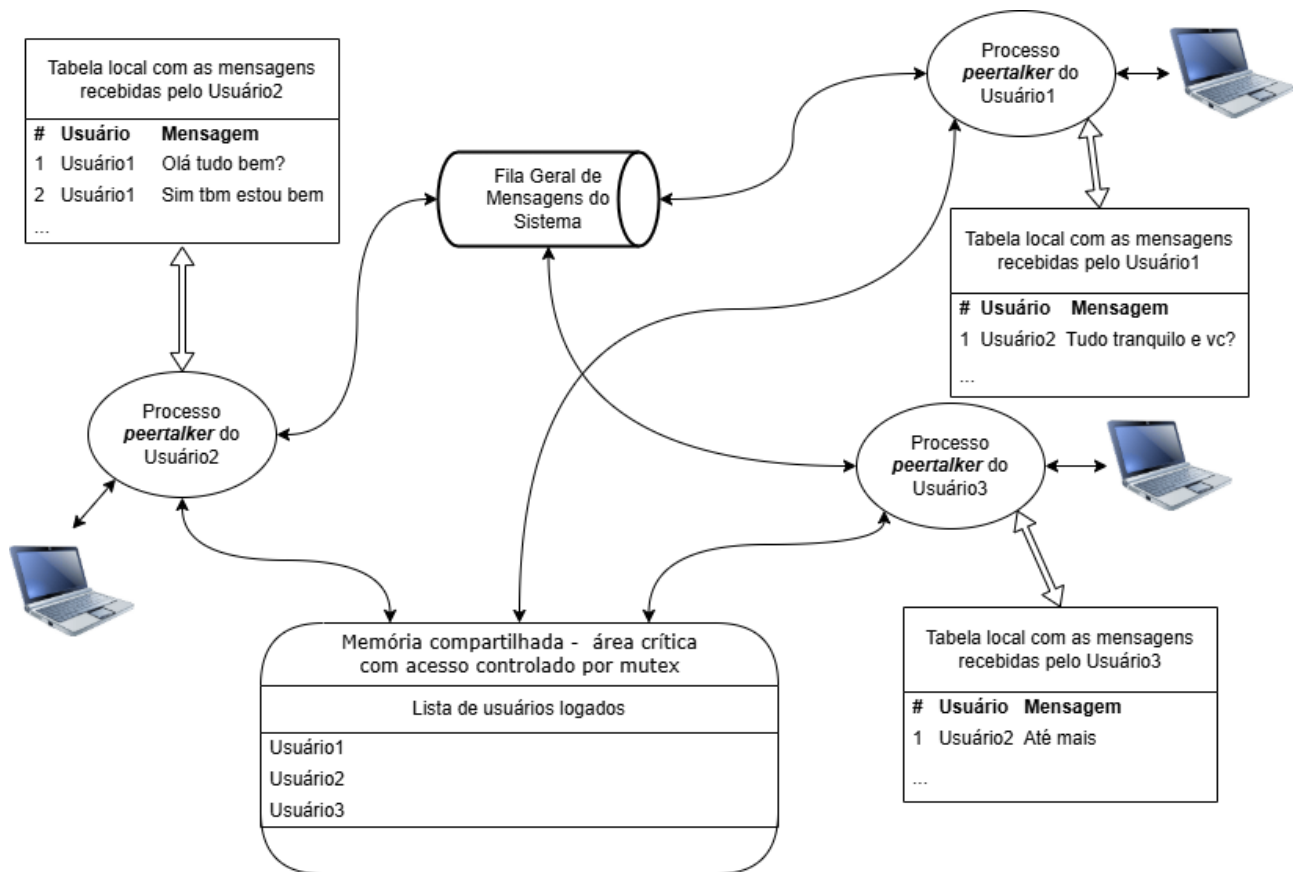


Fig. 4. Diagrama Geral da Arquitetura do Sistema **PeerTalk**

A comunicação entre estes processos *peertalker* é implementada pelos mecanismos de IPC vistos em aula, necessitando das filas de mensagens, de **memória compartilhada** e de **semáforos/mutexes** para implementar corretamente o suporte para a comunicação entre os usuários.

Note que diferente do mecanismo de comunicação de rede utilizado na Internet (a interface *socket*), que segue os princípios da arquitetura cliente-servidor, os mecanismos de IPC de memória compartilhada e filas de mensagens (e também os mecanismos de sincronismo de semáforos e *mutexes*) são essencialmente *peer-to-peer*, não obrigando que os processos tenha que assumir o papel de clientes ou de servidores para poder usar esses mecanismos. Em outras palavras, nada impede que um sistema que use os mecanismos de IPC vistos em aula seja projetado como um sistema cliente-servidor (como o sistema **CentralTalk** sugerido como outro projeto prático), mas nada obriga a usar apenas esse tipo de arquitetura de software para sistemas concorrente, outros tipos de arquitetura também podem ser facilmente explorados, como é o caso da arquitetura *peer-to-peer* usado no sistema **PeerTalk**.

Para entrar no sistema **PeerTalk** um usuário deve abrir uma janela de terminal e executar o programa *peertalker* e se logar no sistema. No login o usuário deve informar seu nome de usuário (não é necessário senha), que é um texto sem espaço em branco de até 20 caracteres.

A lista de usuários logados no sistema deve ser guardada em uma área de memória compartilhada por todos os processos *peertalker*. Essa área é uma seção crítica do sistema **PeerTalk**, assim o acesso mutuamente exclusivo a essa área deve ser garantido por meio de *mutexes*, para evitar que ocorram problemas e conflitos no acesso e (principalmente) na alteração lista de usuários logados. Depois de conseguir acesso exclusivo a lista de usuários logados o processo *peertalker* deve verificar se o usuário que está se logando já não está presente no sistema. Se não houver nenhum outro usuário já logado com o mesmo nome, o login será aceito. Neste caso o processo *peertalker* irá cadastrar o nome do usuário em conjunto com o PID (*Process Identifier*) do processo cliente *peertalker* que servirá como identificador único do usuário enquanto estiver logado. Depois basta liberar o acesso exclusivo para essa lista.

Para troca de informações entre os processos *peertalker* que foram criados para cada usuário distinto logado no sistema é utilizado uma fila de mensagens geral do sistema **PeerTalk**. O campo inicial da mensagem

contém o identificador ou código do tipo de mensagem (**msgtyp**) que é usado para separar as mensagens destinadas a um particular processo *peertalker*. Este campo deve conter o PID do processo *peertalker* a que se destina a mensagem. Logo após o campo inicial com PID do processo *peertalker* de destino, a mensagem deve conter um campo com o PID do processo *peertalker* originador da mensagem. Depois desses dois campos seguem os demais campos de dados da mensagem. Porém com esses dois campos iniciais é possível estabelecer a comunicação *peer-to-peer* entre quaisquer dois processos *peertalker*.

Note que diferente do cliente *speaker* do sistema **CentralTalk**, o processo *peertalker* faz muito mais do que apenas ler os comandos do usuário e verificar se estão corretamente digitados. O processo *peertalker* deve efetivamente executar os comandos com base nas informações contidas nas áreas de memória e nas filas de mensagens compartilhadas entre todos os processos do sistema **PeerTalk**.

Os comandos de usuário tratados pelos processos *peertalker*, são os seguintes:

- **send** <nome-usuário>, <texto-mensagem> – envia uma mensagem direta do usuário atual para o usuário destino identificado por <nome-usuário>. Primeiro o processo *peertalker* deve consultar a lista de usuários logados e verificar se o usuário <nome-usuário> existe. Em caso afirmativo, então uma mensagem contendo <texto-mensagem> e o PID desse usuário (que foi extraído da lista de usuários) deve ser enviada para a fila de mensagens geral do sistema. A mensagem somente é enviada se o usuário estiver logado no sistema, caso contrário uma mensagem de erro deve ser mostrada na tela.
- **recv** – retira da fila de mensagens do sistema, todas as mensagens que tem o PID igual ao PID do usuário corrente e guarda essas mensagens no fim da tabela local de mensagens que já foram recebidas pelo usuário. Se não houver nenhuma mensagem destinada ao usuário avisa que não há mensagens recebidas.
- **msgs** – mostra na tela todas as mensagens que estão na tabela local de mensagens. Se não houver nenhuma mensagem na tabela local, avisa que não há mensagens armazenadas.
- **del msgs** – remove (deleta) todas as mensagens destinadas ao usuário. No caso apenas deleta a cópia local das mensagens que foram enviadas para o usuário, mensagens que ainda estejam na fila de mensagens geral do sistema são deixadas lá e não são excluídas.
- **users** – mostra na tela a lista de usuários logados no sistema. A lista de usuários é mantida como uma estrutura de dados em uma área de memória compartilhada formando uma área crítica. Assim o acesso a essa lista deve ser feito somente através de *mutexes*.
- **myid** – mostra o identificador do usuário atual.
- **exit** – sai do sistema *PeerTalk*, mas antes desloga o usuário, ou seja, descadastra o usuário da lista de usuários logados.